

Resumen del Capítulo 4: Lógica Matemática.

Este capítulo introduce los fundamentos del razonamiento lógico, una herramienta indispensable en la computación para el diseño de circuitos, la creación de algoritmos, la verificación de programas y la inteligencia artificial.

Proposiciones y Operadores Lógicos.

- **Proposición:** Es un enunciado declarativo que puede ser verdadero (V) o falso (F), pero no ambos a la vez. Se representan con letras minúsculas (p, q, r, s).
- **Operadores (Conectivos) Lógicos:** Permiten combinar proposiciones simples para formar proposiciones compuestas:
 - **Negación ($\neg$ o $\sim$):** Invierte el valor de verdad.
 - **Conjunción ($\wedge$ / Y):** Es verdadera solo si ambas proposiciones son verdaderas.
 - **Disyunción ($\vee$ / O):** Es verdadera si al menos una de las proposiciones es verdadera.
 - **Implicación o Condicional ($\rightarrow$):** Indica una relación de causa-efecto ("si p , entonces q "). Solo es falsa cuando el antecedente (p) es verdadero y el consecuente (q) es falso.
 - **Bicondicional o Doble Implicación ($\leftrightarrow$):** Es verdadera si ambas proposiciones tienen el mismo valor de verdad (" p si y solo si q ").

Tablas de Verdad y Clasificación.

Las tablas de verdad son representaciones gráficas que muestran todos los posibles valores de verdad de una proposición compuesta según las combinaciones de sus componentes. Las expresiones se clasifican en:

- **Tautología:** La proposición compuesta es verdadera para cualquier combinación de valores (esencial para demostrar teoremas).
- **Contradicción:** La proposición es siempre falsa.
- **Contingencia:** El resultado final contiene tanto valores verdaderos como falsos.

Equivalencias Lógicas y Álgebra de Proposiciones.

Dos proposiciones son lógicamente equivalentes ($\equiv$) si tienen la misma tabla de verdad. El capítulo detalla las leyes algebraicas para simplificar expresiones lógicas complejas:

- **Leyes de De Morgan:** $\neg(p \wedge q) \equiv \neg p \vee \neg q$ y $\neg(p \vee q) \equiv \neg p \wedge \neg q$.
- **Leyes de Idempotencia, Conmutativas, Asociativas y Distributivas.**

Razonamiento Válido e Inferencia.

Se estudian las reglas de inferencia para determinar la validez de un argumento (demostrar que si las premisas son verdaderas, la conclusión también lo es). Destacan métodos como el *Modus Ponens*, *Modus Tollens* y el silogismo hipotético.

Lógica de Predicados (Cuantificadores).

Dado que la lógica proposicional no puede analizar la estructura interna de las oraciones, se introduce la lógica de predicados mediante variables y cuantificadores:

- **Cuantificador Universal ($\forall$):** "Para todo x ".
- **Cuantificador Existencial ($\exists$):** "Existe al menos un x ".

Resumen del Capítulo 5: Álgebra Booleana y Circuitos Lógicos.

Este capítulo actúa como el puente directo entre la lógica matemática teórica y el hardware real de las computadoras, mostrando cómo se manipulan los bits (0 y 1) a nivel físico.

Es una estructura algebraica que define operaciones binarias sobre un conjunto de elementos que usualmente toman los valores $\{0, 1\}$. En computación, el 1 representa "verdadero", "encendido" o "voltaje alto", mientras que el 0 representa "falso", "apagado" o "voltaje bajo".

Funciones y Expresiones Booleanas.

Una función booleana toma variables binarias como entrada y produce un resultado binario. Estas funciones se pueden expresar de dos formas normalizadas para su análisis:

- **Suma de Productos (Forma Disyuntiva - Minterminos):** Combinaciones que hacen que la función sea \$1\$.
- **Producto de Sumas (Forma Conjuntiva - Maxterminos):** Combinaciones que hacen que la función sea \$0\$.

Circuitos Combinatorios y Compuertas Lógicas.

Las operaciones del álgebra booleana se implementan físicamente en hardware mediante **compuertas lógicas**. El capítulo describe el comportamiento y simbología de:

- **Compuerta AND (Multiplicación booleana):** Produce salida \$1\$ solo si todas las entradas son \$1\$.
- **Compuerta OR (Suma booleana):** Produce salida \$1\$ si al menos una entrada es \$1\$.
- **Compuerta NOT (Inversor):** Cambia el estado de la entrada (\$1 \rightarrow 0\$; \$0 \rightarrow 1\$).
- **Compuertas Compuestas:** NAND (AND negada), NOR (OR negada) y XOR (OR exclusiva, clave para operaciones aritméticas).

Simplificación de Funciones Booleanas.

En el diseño de hardware, simplificar una función booleana es crucial para reducir el número de compuertas lógicas necesarias, disminuyendo costos de fabricación y consumo de energía. El autor explica dos métodos principales:

- **Método Algebraico:** Aplicando los teoremas y postulados del álgebra de Boole (identidad, complemento, distribución, absorción).
- **Mapas de Karnaugh (Mapas K):** Un método gráfico sumamente eficiente para simplificar expresiones de 2, 3, 4 o más variables mediante la agrupación de unos (\$1\$) contiguos en una cuadrícula.